

Relatório Técnico

Mini E-commerce Distribuído — Arquitetura de Microsserviços

1. Comunicação entre Serviços

Estratégia implementada. Todos os serviços comunicam-se via HTTP/REST síncrono. O API Gateway atua como proxy reverso: intercepta cada requisição, verifica o registry interno e repassa com headers preservados (Authorization, Content-Type) mais o marcador X-Gateway-Request: true. A única comunicação serviço-a-serviço fora do gateway ocorre em dois pontos: (a) Orders consulta Products via GET /products/:id para validar existência antes de persistir o pedido; (b) a primária de Products envia POST /internal/sync à réplica, protegido por X-Internal-Key.

Trade-offs: REST vs Filas de Mensagem

Critério	REST síncrono (implementado)	Fila de mensagens
Acoplamento	Forte — Orders depende de Products estar UP	Fraco — producer publica e prossegue
Latência percebida	Imediata — resposta inclui validação	Assíncrona — ACK antes da consistência final
Tolerância a falhas	Products down → Orders retorna 502	Orders persiste mesmo com Products indisponível
Rastreabilidade	Trivial — um request-id por chamada	Requer correlation IDs e tracing distribuído
Complexidade operacional	Baixa	Alta — broker, DLQ, consumers

Para catálogos com baixa taxa de mutação e validação crítica de existência, REST síncrono é adequado. Em alto volume, um cache local em Orders (TTL curto) eliminaria a dependência em tempo de escrita sem a complexidade de um broker.

2. Consistência na Replicação de Produtos

Estratégia: primary-backup síncrono com degradação graceful. A escrita segue cinco etapas: (1) cliente envia POST /products ao gateway, roteado à primária (5002); (2) primária persiste localmente; (3) **antes de responder**, envia POST /internal/sync à réplica com timeout de 3 s; (4) réplica confirma → resposta 201 Created; (5) réplica falha → resposta 201 Created com header X-Replica-Warning: replica sync failed.

Justificativa. O risco real é vender um item inexistente — por isso a escrita deve ser atômica na primária antes de qualquer confirmação ao cliente. A degradação graceful evita que falha temporária da réplica bloqueie criações. DynamoDB nomeia modelo similar de *strong consistency reads + eventual replication*. **Limitação:** produtos criados durante indisponibilidade da réplica não são reenviados após recuperação — replication log com replay seria necessário para garantir convergência.

3. Falha do Serviço de Pedidos

O heartbeat verifica /health a cada 5 s (timeout 2 s). Após **2 falhas consecutivas** o serviço é marcado "down" no registry. Requisições para /orders/* retornam imediatamente 503 Service Unavailable sem tentativa de conexão. Users e Products continuam **100% operacionais** — não há dependência deles com Orders em tempo de leitura ou escrita. O gateway registra [FAILURE] orders caiu em <timestamp>.

Risco de perda de dados em trânsito. Pedidos que chegam exatamente no instante da falha — após parsing do body mas antes da gravação em disco — são perdidos silenciosamente. Mitigações para produção:

- **Idempotência por chave de cliente:** idempotency-key UUID no header; duplicatas rejeitadas com 409, permitindo retentativas seguras.
- **Write-ahead log (WAL):** requisição gravada antes do processamento; processo de recovery reaplica entradas não confirmadas após reinicialização.
- **Fila durável:** gateway publica em fila persistente (RabbitMQ durable: true); serviço confirma (ack) somente após persistência bem-sucedida.

4. Autenticação JWT e Controle de Acesso por Role

- [1] POST /users/login → bcrypt.compare(password, hash)
- [2] JWT assinado: { userId, email, role, exp: +2h } → token ao cliente
- [3] Cliente: Authorization: Bearer <token> → gateway repassa intacto
- [4] Serviço: jwt.verify(token, JWT_SECRET) → payload em req.user
- [5] requireAdmin: req.user.role !== 'admin' → 403 Forbidden

O JWT carrega role: "user" assinado com JWT_SECRET (HMAC-SHA256). O middleware requireAdmin rejeita a requisição antes de qualquer acesso ao body. Se o cliente forja o payload do token, a assinatura deixa de bater e jwt.verify lança JsonWebTokenError, capturado como 401. O segredo nunca trafega entre serviços — cada um valida independentemente com a mesma variável de ambiente, eliminando um auth service centralizado ao custo de exigir lista negra distribuída para revogação imediata de tokens.

5. Limitações em Relação a um Ambiente de Produção

Limitação	Impacto	Solução em Produção
Gateway como SPOF	Queda total do sistema	Multi-instância + load balancer (NGINX/ALB)
Sem circuit breaker	Falhas em cascata por latência	opossum / Resilience4j com half-open state
Sem service discovery	URLs hardcoded em env vars	Consul, Kubernetes DNS, AWS Service Connect
Sync sem replay	Divergência permanente pós-falha	Replication log + reconciliação periódica
Sem retry automático	Falha transiente = erro permanente	Retry exponencial com jitter no gateway
Sem rate limiting	Vulnerável a abuso e DDoS	express-rate-limit / API Gateway gerenciado
Segredos em env vars	Exposição em logs / ps aux	AWS Secrets Manager / HashiCorp Vault
Sem TLS interno	Tráfego plaintext entre serviços	mTLS com cert-manager / Istio / Linkerd
Persistência em JSON	Sem transações nem concorrência segura	PostgreSQL/MongoDB com replicação nativa
Sem startup healthcheck	Gateway roteia antes do serviço estar pronto	healthcheck + condition: service_healthy

A arquitetura implementada atende os requisitos de um ambiente de demonstração com separação clara de responsabilidades, replicação funcional e detecção de falhas. As limitações elencadas não são defeitos de design, mas simplificações deliberadas — cada uma possui contrapartida conhecida em sistemas de produção, onde os trade-offs de custo operacional, complexidade e resiliência são avaliados conforme a criticidade do domínio.